



J D College Of Engineering & Management

Unit-2

Unit Testing

Dr. A. R. Welekar
Associate Professor(CSE)

❖ Concepts of Unit Testing:

Unit testing is the process of testing the smallest parts of your code, like it is a method in which we verify the code's correctness by running one by one. It's a key part of software development that improves code quality by testing each unit in isolation.

What is a Unit Test:

- A unit test is a small piece of code that checks if a specific function or method in an application works correctly. It will work as the function inputs and verifying the outputs. These tests check that the code works as expected based on the logic the developer intended.
- In these multiple tests are written for a single function to cover different possible scenarios and these are called test cases. While it is ideal to cover all expected behaviors, it is not always necessary to test every scenario.
- Unit tests should run one by one, it means that they do not depend on other system parts like databases or networks. Instead, data stubs can be used to simulate these dependencies. Writing unit tests is easiest for simple, self-contained code blocks.

Unit Testing Strategies :

To create effective unit tests, follow these basic techniques to ensure all scenarios are covered:

- Logic checks: Verify if the system performs correct calculations and follows the expected path with valid inputs. Check all possible paths through the code are tested.
- Boundary checks: Test how the system handles typical, edge case, and invalid inputs. For example, if an integer between 3 and 7 is expected, check how the system reacts to a 5 (normal), a 3 (edge case), and a 9 (invalid input).
- Error handling: Check the system properly handles errors. Does it prompt for a new input, or does it crash when something goes wrong?
- Object-oriented checks: If the code modifies objects, confirm that the object's state is correctly updated after running the code.

Types of Unit Testing :

1. Manual unit testing:

Manual Testing is like checking each part of a project by hand, without using any special tools. People, like developers, do each step of the testing themselves. But manual unit testing isn't used much because there are better ways to do it and it has some problems:

- It costs more because workers have to be paid for the time they spend testing, especially if they're not permanent staff.
- It takes a lot of time because tests have to be done every time the code changes.
- It is hard to find and fix problems because it is tricky to test each part separately.
- Developers often do manual testing themselves to see if their code works correctly.

2. Automated unit testing:

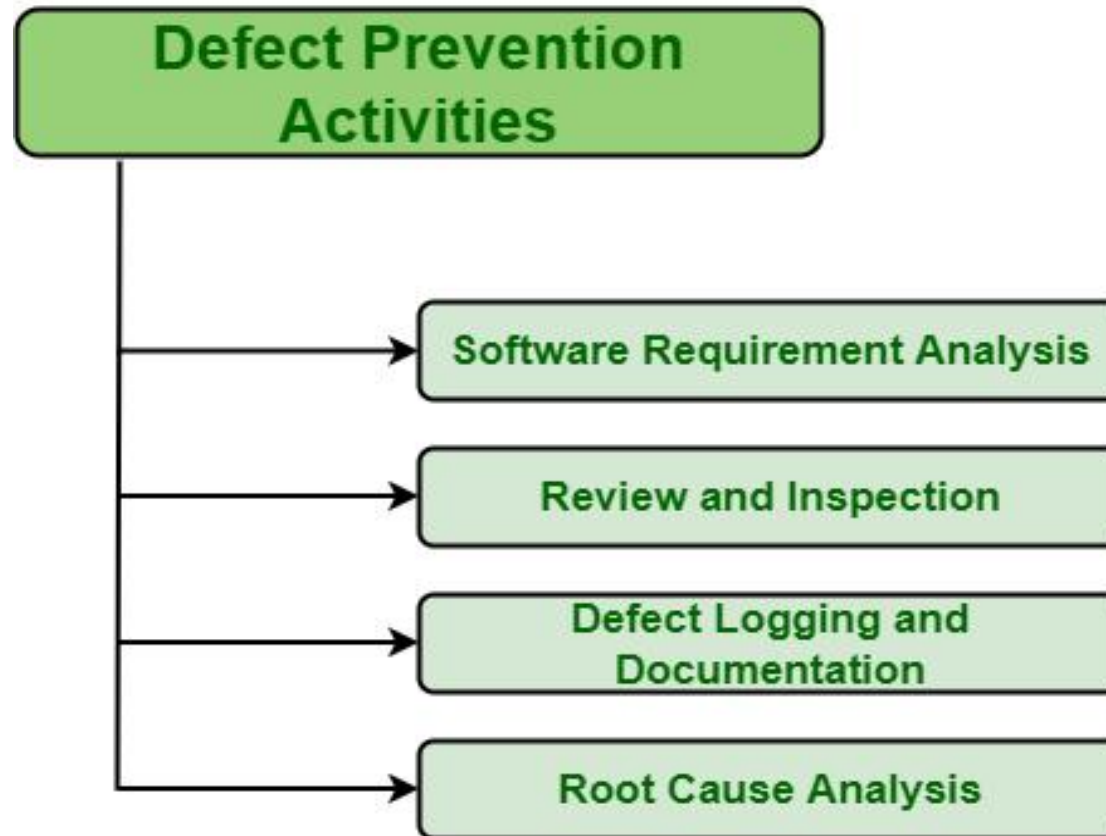
Automation Unit Testing is a way of checking if software works correctly without needing lots of human effort. We use special tools made by people to run these tests automatically. These are part of the process of building the software. Here's how it works:

- Developers write a small piece of code to test a function in the software. This code is like a little experiment to see if everything works as it should.
- Before the software is finished and sent out to users, these test codes are taken out. They're only there to make sure everything is working properly during development.
- Automated testing can help us check each part of the software on its own. This helps us find out if one part depends too much on another. It's like putting each piece of a puzzle under a magnifying glass to see if they fit together just right.
- We usually use special tools or frameworks to do this testing automatically. These tools can tell us if any of our code doesn't pass the tests we set up.
- The tests we write should be really small and focus on one specific thing at a time. They should also run on the computer's memory and not need internet connection.

❖ Defect Prevention:

- It is basically defined as a measure to ensure that defects being detected so far, should not appear or occur again.
- For facilitating communication simply among members of team, planning and devising defect prevention guidelines, etc., Coordinator is mainly responsible.
- Coordinator is mainly responsible to lead defect prevention efforts, to facilitate meetings, to facilitate communication between team members and management, etc.
- DP board generally has a quarterly plan in which sets some goals at organization level. To achieve these goals, various methods or activities are mostly used and carried out to achieve and complete these goals.

Methods of Defect Prevention :



1. Software Requirement Analysis :

- The main cause of defects in software products is due to error in software requirements and designs. Software requirements and design both are important, and should be analyzed in an efficient way with more focus.
- Software requirement is basically considered an integral part of Software Development Life Cycle (SDLC). These are the requirements that basically describes features and functionalities of target product and also conveys expectations or requirement of users from software product.
- Therefore, it is very much needed to understand about software requirements more carefully, If requirements are not understood well by tester and developers, then there might be chance of occurring of issue or defect in further process. Therefore, it is essential to analyze and evaluate requirements in more appropriate and proper manner.

2. Review and Inspection :

- Review and inspection, both are essential and integral part of software development.
- They are considered as powerful tools that can be used to identify and remove defects if present before their occurrence and impact on production.
- Review and inspection come in different levels or stages of defect prevention to meet different needs.
- They are used in all software development and maintenance methods. There are two types of review i.e., self-review and peer-review.

3. Defect Logging and Documentation :

- After successful analysis and review, there should be records maintained about defects to simply complete description of defect.
- This record can be further used to have better understanding of defects.
- After getting knowledge and understanding of defect, then only one can take some effective and required measures and actions to resolve particular defects so that defect cannot be carried further to next phase.

4. Root Cause Analysis :

- Root because analysis is basically analysis of main cause of defect. It simply analysis what triggered defect to occur.
- After analyzing main cause of defect, one can find best way to simply avoid occurrence of such types of defects next time.

5. Static Code Analysis :

- To find any problems in the source code without running the program, use automated techniques for static code analysis.
- Before the code is even compiled, these tools can detect typical programming errors, coding standard violations and other problems.

6. Pair programming :

- It involves two programmers sharing a workstation, where one writes code while the other goes over each line as it is written.
- This real-time communication between team members promotes constant feedback and helps in the early detection of mistakes

7. Test-Driven Development (TDD) :

- Prior to writing any code, write automated tests.
- This method helps identify errors early in the development process and guarantees that the code complies with the requirements.

8. Training and Skill Development :

- Investing in continuous training and skill development for team members is recommended.
- Skilled developers are better able to follow best practices and are less prone to make frequent mistakes.

9. Checklists :

- To make sure that crucial actions are not missed during the various stages of development, use checklists.
- Developers and reviewers can quickly confirm that established standards are being followed by using checklists as a reference.

❖ Static Unit Testing:

- Static Testing is a type of Software Testing method that is performed to check the defects in software without actually executing the code of the software application.
- Also known as Verification testing or Non-execution testing is a type of Software Testing method that is performed in the early stage of development to avoid errors as it is easier to find sources of failures and it can be fixed easily.

❖ Static Unit Testing:

Need for Static Testing:

- Static testing is needed whenever the following situations are encountered while testing an application or software:
- **Increased software size:** Static testing is required to get free from bugs in the early stages of development as with testing activity the size of the software increases which is difficult to handle due to a reduction in the productivity of the code coverage.
- **Dynamic testing is expensive:** Dynamic testing is more expensive than static testing as dynamic testing uses test cases that have been created in the initial stages and there is also a need to preserve the implementation and validation of the test cases which takes a lot of time from test engineers:

❖ Static Unit Testing:

Need for Static Testing:

- **Dynamic testing is time-consuming:** Static testing is required as dynamic testing is a time-consuming process.
- **Bugs detection at early stages:** Static testing is helpful as it finds bugs at early stages, while dynamic testing finds bugs at later stages which makes it time-consuming and costly to fix the bugs.
- **Improvement of development productivity:** Static testing helps to identify bugs early in the software development thus it helps to reduce the flaws during production and increase development productivity.

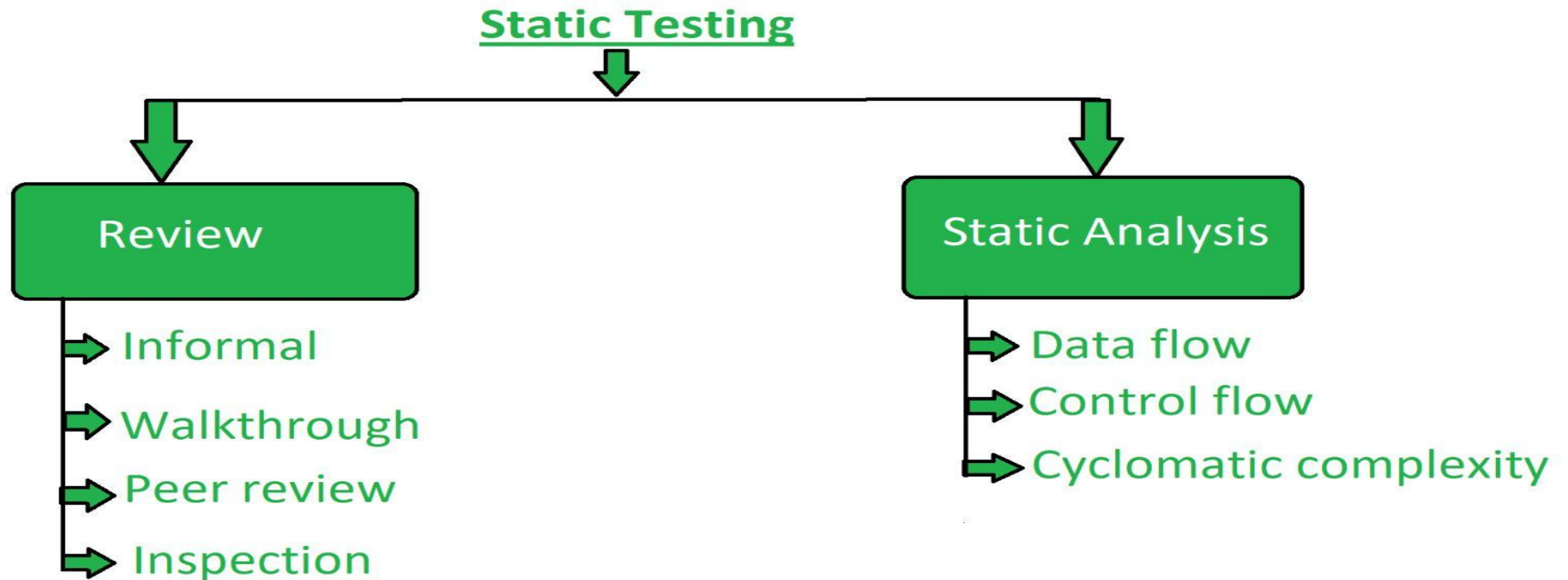
❖ Static Unit Testing:

Objectives of Static Testing:

- **Decreases Flaws:** Static testing will decrease the flaws in production as the bugs will be detected early in the software development.
- **Saves Time:** Early detection of the bugs helps to save a lot of time, effort, and cost that will be required to fix the bugs.
- **Easy Bug Fixing:** Static testing is used to identify the bugs early in software development, where it is quite easy to fix the bugs.
- **Quality Improvement:** Enhance overall software quality of the software product by ensuring the compliance with the coding standards and best practices.
- **Cost Efficiency:** Static testing helps to reduce the cost associated with dynamic testing by catching defects early.

❖ Static Unit Testing:

Static Testing Techniques:



❖ Static Unit Testing:

1. Review:

In static testing, the review is a process or technique that is performed to find potential defects in the design of the software. It is a process to detect and remove errors and defects in the different supporting documents like software requirements specifications. Review is of four types:

- **Informal:** In an informal review the creator of the documents put the contents in front of an audience and everyone gives their opinion and thus defects are identified in the early stage.
- **Walkthrough:** It is basically performed by an experienced person or expert to check the defects so that there might not be problems further in the development or testing phase.
- **Peer review:** Peer review means checking documents of one another to detect and fix defects. It is basically done in a team of colleagues.
- **Inspection:** Inspection is basically the verification of documents by the higher authority like the verification of software requirement specifications (SRS).

❖ Static Unit Testing:

2. Static Analysis:

Static Analysis includes the evaluation of the code quality that is written by developers. Different tools are used to do the analysis of the code and comparison of the same with the standard. It also helps in following identification of the following defects:

1. Unused variables.
2. Dead code.
3. Infinite loops.
4. Variable with an undefined value.
5. Wrong syntax.

❖ Static Unit Testing:

2. Static Analysis:

Static Analysis is of three types:

1. **Data Flow:** Data flow is related to the stream processing.
2. **Control Flow:** Control flow is basically how the statements or instructions are executed.
3. **Cyclomatic Complexity:** Cyclomatic complexity defines the number of independent paths in the control flow graph made from the code or flowchart so that a minimum number of test cases can be designed for each independent path.

❖ Static Unit Testing:

How Static Testing is Performed?

Static Analysis is of three types:

1. **Data Flow:** Data flow is related to the stream processing.
2. **Control Flow:** Control flow is basically how the statements or instructions are executed.
3. **Cyclomatic Complexity:** Cyclomatic complexity defines the number of independent paths in the control flow graph made from the code or flowchart so that a minimum number of test cases can be designed for each independent path.

❖ **Dynamic Testing:**

- Dynamic Testing is a type of Software Testing that is performed to analyze the dynamic behavior of the code. It includes the testing of the software for the input values and output values that are analyzed.
- It is performed to describe the dynamic behavior of code. It refers to the observation of the physical response from the system to variables that are not constant and change with time.

❖ Dynamic Testing:

Objective of Dynamic Testing:

- **Find errors and bugs:** Through comprehensive testing, find and expose flaws, faults, or defects in the software code and its functionality so that they can be fixed as soon as possible.
- **Verify the behavior of the system:** Verify that the software operates as expected and complies with company requirements, industry or regulatory standards, user expectations, and any applicable business regulations.
- **Assessing Performance:** To make sure the software satisfies performance requirements, evaluate its performance by monitoring reaction times, throughput, and use of resources under various scenarios.

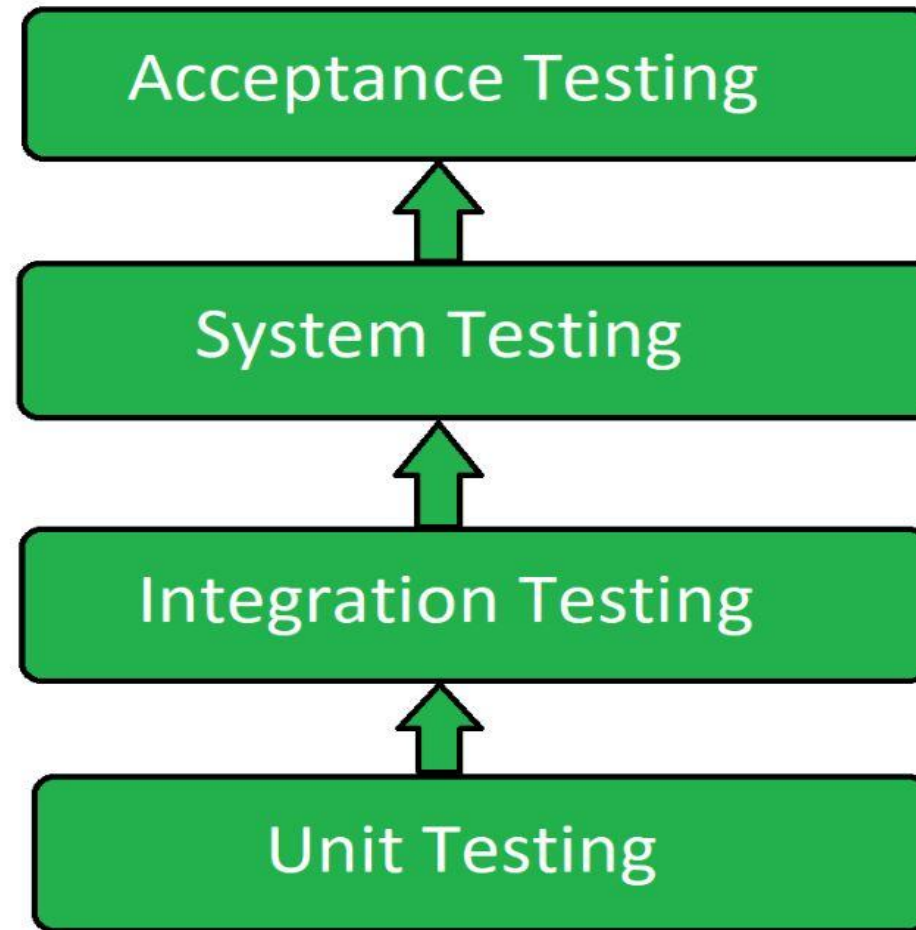
❖ **Dynamic Testing:**

Objective of Dynamic Testing:

- **Assure Trustworthiness:** Examine the software's dependability by determining how well it performs regularly under typical operating conditions, free of unexpected faults or crashes.
- **Accuracy of Test Data:** Verify the precision and consistency of the data handled by the software to guarantee reliable and uniform information handling.
- **Assess Scalability:** Examine whether the application can grow to handle more users, workloads, or data volumes without seeing an obvious decline in performance.

❖ Dynamic Testing:

Levels of Dynamic Testing:



❖ Dynamic Testing:

Levels of Dynamic Testing:

1. **Unit testing:** Unit testing is the process of testing individual software components or "units" of code to ensure that they are working as intended. Unit tests are typically small and focus on testing a specific feature or behavior of the software.
2. **Integration testing:** Integration testing is the process of testing how different components of the software work together. This level of testing typically involves testing the interactions between different units of code, and how they function when integrated into the overall system.
3. **System testing:** System testing is the process of testing the entire software system to ensure that it meets the specified requirements and is working as intended. This level of testing typically involves testing the software's functionality, performance, and usability.

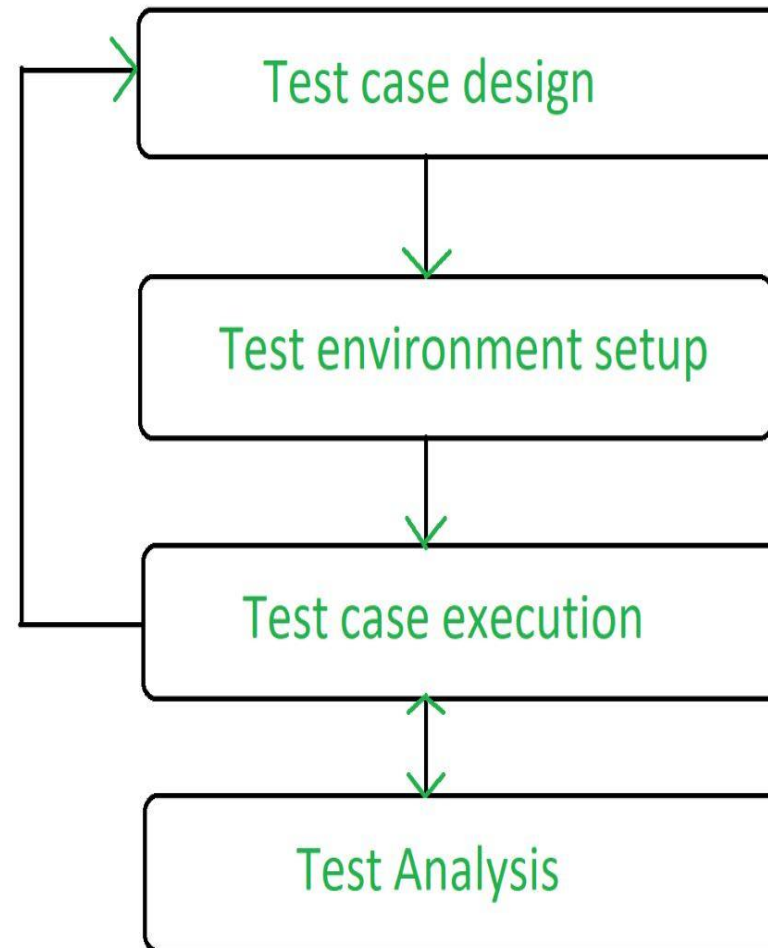
❖ Dynamic Testing:

Levels of Dynamic Testing:

4. **Acceptance testing:** Acceptance testing is the final stage of dynamic testing, which is done to ensure that the software meets the needs of the end-users and is ready for release. This level of testing typically involves testing the software's functionality and usability from the perspective of the end-user.
5. **Performance testing:** Performance testing is a type of dynamic testing that is focused on evaluating the performance of a software system under a specific workload. This can include testing how the system behaves under heavy loads, how it handles a large number of users, and how it responds to different inputs and conditions.
6. **Security testing:** Security testing is a type of dynamic testing that is focused on identifying and evaluating the security risks associated with a software system. This can include testing how the system responds to different types of security threats, such as hacking attempts, and evaluating the effectiveness of the system's security features.

❖ Dynamic Testing:

Dynamic Testing Process Phase:



❖ Dynamic Testing:

Dynamic Testing Process Phase:

1. **Test Case Design:** It defines the test objectives, scope and criteria. It defines test data and expected outcomes and develops test cases based on requirements and specifications. It generates test cases that address various programmes features.
2. **Test Environment Setup:** It sets up the settings and infrastructure required for testing. It configured the network, hardware and software in the test environment. Additionally, it makes sure that the test environment matches the production environment by installing and configuring the required test tools and test harnesses.

❖ Dynamic Testing:

Dynamic Testing Process Phase:

3. **Test Case Execution:** Using the specified test data, it runs the test cases in order to verify the software's behavior. It keeps track of and logs the actual outcomes, comparing them with the predicted results to find any differences. It runs test scenarios in both positive and negative modes.
4. **Test Analysis:** It evaluates the general behavior of the system and finds faults by analyzing the test case outcomes. Any inconsistencies or flaws discovered during test execution are documented and reported. It works along with development teams to figure out and address concerns that are reported.

❖ Mutation Testing:

- Mutation Testing is a white box Testing technique that helps compare the effectiveness of existing tests and design new ones.
- Making small modifications (mutations) in the source code, mutation testing aims to identify weaknesses in test cases and verify that all parts of the program are properly tested with their functionalities.
- Mutation Testing was introduced by Richard Lipton in 1971. While the high cost initially limited its use, but now it is widely used in languages like Java and XML, proving its effectiveness in software testing.

❖ Mutation Testing:

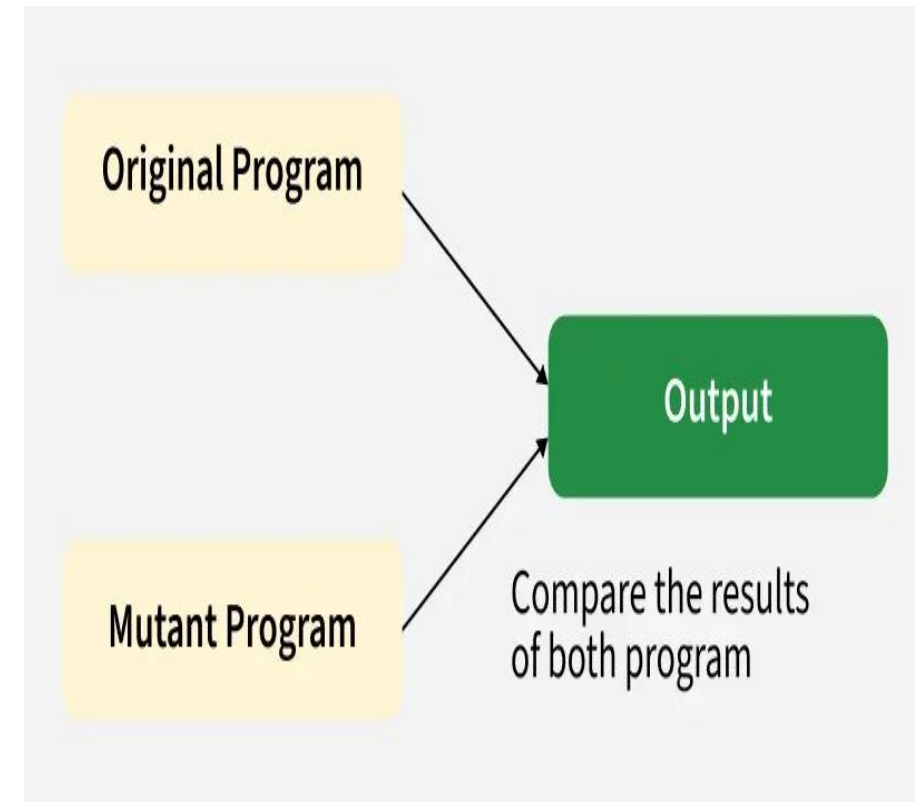
The objective of mutation testing is:

- To identify pieces of code that are not tested properly.
- To identify hidden defects that can't be detected using other testing methods.
- To discover new kinds of errors or bugs.
- To calculate the mutation score.
- Understand how errors spread within the program.
- Assess how well the test cases perform.

❖ Mutation Testing:

The objective of mutation testing is:

- To identify pieces of code that are not tested properly.
- To identify hidden defects that can't be detected using other testing methods.
- To discover new kinds of errors or bugs.
- To calculate the mutation score.
- Understand how errors spread within the program.
- Assess how well the test cases perform.



❖ Mutation Testing:

Types of Mutation Testing:

Mutation testing can be classified into three main types, based on the nature of the code changes:

1. Value Mutations
2. Decision Mutations
3. Statement Mutations:

❖ Mutation Testing:

1. Value Mutations:

In this type of testing the values are changed to detect errors in the program. Basically a small value is changed to a larger value or a larger value is changed to a smaller value. In this testing basically constants are changed.

Initial Code:

```
int mod = 1000000007;
```

```
int a = 12345678;
```

```
int b = 98765432;
```

```
int c = (a + b) % mod;
```

Changed Code:

```
int mod = 1007;
```

```
int a = 12345678;
```

```
int b = 98765432;
```

```
int c = (a + b) % mod;
```

❖ Mutation Testing:

2. Decision Mutations:

In decisions mutations are logical or arithmetic operators are changed to detect errors in the program.

Initial Code:

```
if(a < b)
```

```
    c = 10;
```

```
else
```

```
    c = 20;
```

Changed Code:

```
if(a > b) // Mutation: Reversed the  
condition
```

```
    c = 10;
```

```
else
```

```
    c = 20;
```

❖ Mutation Testing:

3. Statement Mutations:

In Statement Mutations, a statement is either deleted or replaced with another statement. This tests how the program handles changes to its flow.

Initial Code:

```
if(a < b)
```

```
    c = 10;
```

```
else
```

```
    c = 20;
```

Changed Code:

```
if(a < b)
```

```
    d = 10; // Mutation: Replaced
```

```
variable
```

```
else
```

```
    d = 20;
```

❖ Mutation Testing:

Tools for Mutation Testing:

In Statement Mutations, a statement is either deleted or replaced with another statement. This tests how the program handles changes to its flow.

Several tools are available for performing mutation testing, including:

- Judy
- Jester
- Jumble
- PIT
- MuClipse.

❖ Debugging:

What is Debugging:

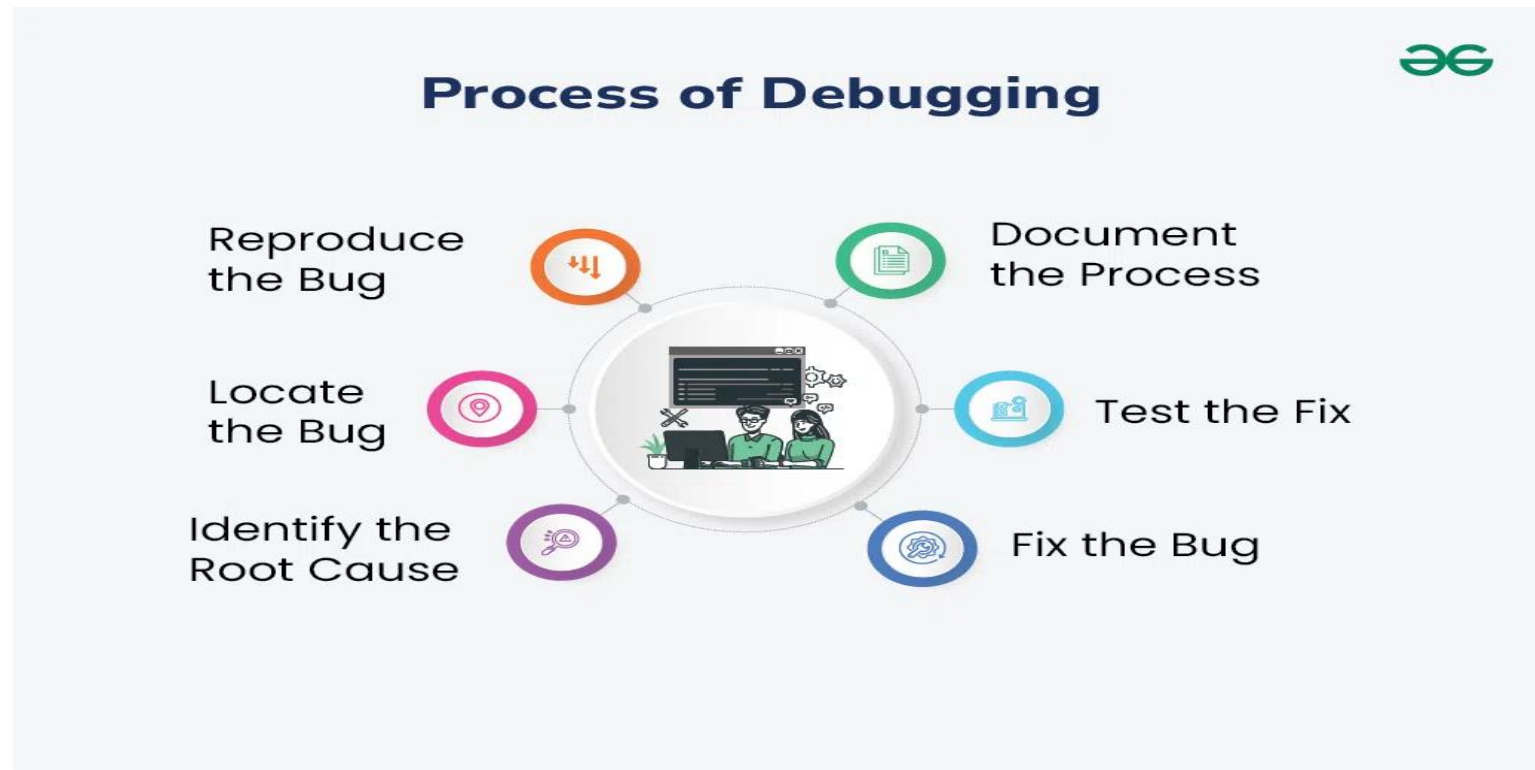
In the context of software engineering, debugging is the process of fixing a bug in the software. When there's a problem with software, programmers analyze the code to figure out why things aren't working correctly. They use different debugging tools to carefully go through the code, step by step, find the issue, and make the necessary corrections.



❖ Debugging:

Process of Debugging:

Debugging is a crucial skill in programming. Here's a simple, step-by-step explanation to help you understand and execute the debugging process effectively:



❖ Debugging:

Step 1: Reproduce the Bug

- To start, you need to recreate the conditions that caused the bug. This means making the error happen again so you can see it firsthand.
- Seeing the bug in action helps you understand the problem better and gather important details for fixing it.

Step 2: Locate the Bug

- Next, find where the bug is in your code. This involves looking closely at your code and checking any error messages or logs.
- Developers often use debugging tools to help with this step.

❖ Debugging:

Step 3: Identify the Root Cause

- Now, figure out why the bug happened. Examine the logic and flow of your code and see how different parts interact under the conditions that caused the bug.
- This helps you understand what went wrong.

Step 4: Fix the Bug

- Once you know the cause, fix the code. This involves making changes and then testing the program to ensure the bug is gone.
- Sometimes, you might need to try several times, as initial fixes might not work or could create new issues.
- Using a version control system helps track changes and undo any that don't solve the problem.

❖ Debugging:

Step 5: Test the Fix

After fixing the bug, run tests to ensure everything works correctly. These tests include:

- **Unit Tests:** Check the specific part of the code that was changed.
- **Integration Tests:** Verify the entire module where the bug was found.
- **System Tests:** Test the whole system to ensure overall functionality.
- **Regression Tests:** Make sure the fix didn't cause any new problems elsewhere in the application

Step 6: Document the Process

- Finally, record what you did. Write down what caused the bug, how you fixed it, and any other important details.
- This documentation is helpful if similar issues occur in the future.

❖ Debugging:

Debugging Approaches/Strategies:

1. **Brute Force:** Study the system for a longer duration to understand the system. It helps the debugger to construct different representations of systems to be debugged depending on the need. A study of the system is also done actively to find recent changes made to the software.
2. **Backtracking:** Backward analysis of the problem which involves tracing the program backward from the location of the failure message to identify the region of faulty code. A detailed study of the region is conducted to find the cause of defects.
3. **Forward analysis** of the program involves tracing the program forwards using breakpoints or print statements at different points in the program and studying the results. The region where the wrong outputs are obtained is the region that needs to be focused on to find the defect.

❖ Debugging:

Debugging Approaches/Strategies:

4. Using A **debugging experience** with the software debug the software with similar problems in nature. The success of this approach depends on the expertise of the debugger.
5. **Cause elimination:** it introduces the concept of binary partitioning. Data related to the error occurrence are organized to isolate potential causes.
6. **Static analysis:** Analyzing the code without executing it to identify potential bugs or errors. This approach involves analyzing code syntax, data flow, and control flow.
7. **Dynamic analysis:** Executing the code and analyzing its behavior at runtime to identify errors or bugs. This approach involves techniques like runtime debugging and profiling.

❖ Debugging:

Debugging Approaches/Strategies:

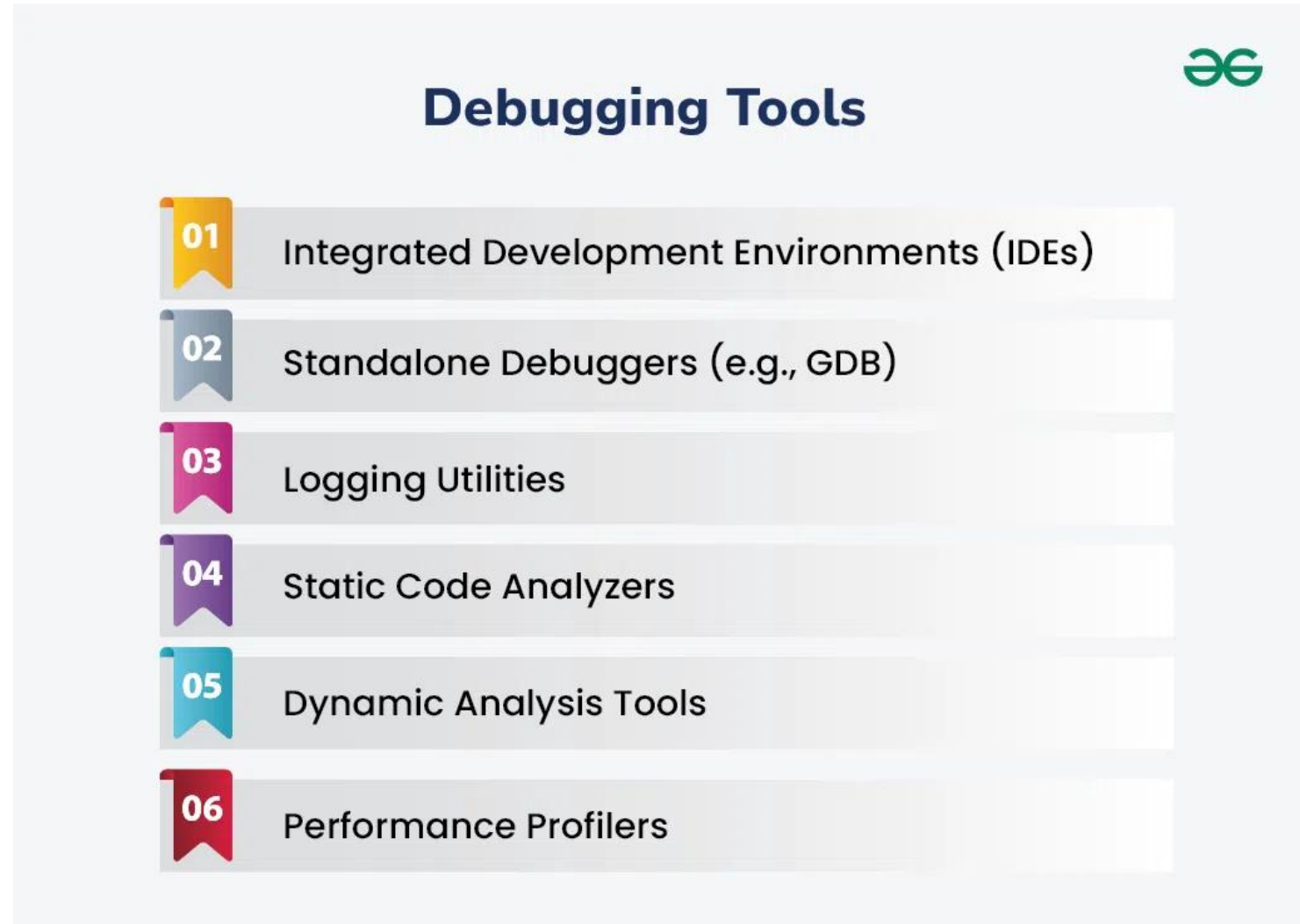
8. **Collaborative debugging:** Involves multiple developers working together to debug a system. This approach is helpful in situations where multiple modules or components are involved, and the root cause of the error is not clear.
9. **Logging and Tracing:** Using logging and tracing tools to identify the sequence of events leading up to the error. This approach involves collecting and analyzing logs and traces generated by the system during its execution.
10. **Automated Debugging:** The use of automated tools and techniques to assist in the debugging process. These tools can include static and dynamic analysis tools, as well as tools that use machine learning and artificial intelligence to identify errors and suggest fixes.

❖ Debugging:

Debugging Tools:

- Debugging tools are essential for software development, helping developers locate and fix coding errors efficiently.
- With the rapid growth of software applications, the demand for advanced debugging tools has increased significantly.
- Companies are investing heavily in these tools, and researchers are developing innovative solutions to enhance debugging capabilities, including AI-driven debuggers and autonomous debugging for specialized applications.
- Debugging tools vary in their functionalities, but they generally provide command-line interfaces to help developers identify and resolve issues. Many also offer remote debugging features and tutorials, making them accessible to beginners. Here are some of the most commonly used debugging tools:

❖ Debugging:



❖ Debugging:

1. Integrated Development Environments (IDEs)

IDEs like Visual Studio, Eclipse, and PyCharm offer features for software development, including built-in debugging tools. These tools allow developers to:

- Execute code line-by-line (step debugging)
- Stop program execution at specific points (breakpoints)
- Examine the state of variables and memory

IDEs support many programming languages and scripting languages, often through open-source plugins.

❖ Debugging:

2. Standalone Debuggers

Standalone debuggers like GDB (GNU Debugger) provide advanced debugging features:

- Conditional breakpoints and watchpoints
- Reverse debugging (running a program backwards)

These tools are powerful but have a steeper learning curve compared to IDE debuggers.

❖ Debugging:

3. Logging Utilities

Logging utilities log a program's state at various points in the code, which can then be analyzed to find problems. Logging is particularly useful for debugging issues that only occur in production environments.

4. Static Code Analyzers

Static code analysis tools examine code without executing it to find potential errors and deviations from coding standards. They focus on the semantics of the source code, helping developers catch common mistakes and maintain consistent coding styles.

❖ Debugging:

5. Dynamic Analysis Tools

Logging utilities log a program's state at various points in the code, which can then be analyzed to find problems. Logging is particularly useful for debugging issues that only occur in production environments.

6. Performance Profilers

Performance profilers help developers identify performance bottlenecks in their code. They measure:

- CPU usage
- Memory usage
- I/O operations

❖ Debugging:

Difference Between Debugging and Testing:

Aspects	Testing	Debugging
Definition	<u>Testing</u> is the process to find bugs and errors.	<u>Debugging</u> is the process of correcting the bugs found during testing.
Purpose	The purpose of testing is to identify defects or errors in the software system	The purpose of debugging is to fix those defects or errors.
Focus	It is the process to identify the failure of implemented code.	It is the process to give absolution to code failure.
Timing	Testing is done before debugging	Debugging Differences between Testing and Debugging is done after testing
Approach	Testing involves executing the software system with test cases	Debugging involves analyzing the symptoms of a problem and identifying the root cause of the problem
Tools and Technique	Testing can involve using automated or manual testing tools	Debugging typically involves using tools and techniques such as logging, tracing, and code inspection.

❖ Tools for Unit Testing:

Unit Testing is a part of software testing where testing is performed on isolated small parts of the code. The objective of Unit Testing is to isolate small portions of application code that could be individually tested and verified, provided all the isolated code units have been developed in the correct procedure.

- It is performed by the development Team in the SDLC process.
- There are a set of valid inputs that would be provided to a unit of code to verify whether the expected values are returned or not.
- At the same time, a set of invalid inputs would also be done to check how the system would handle errors.

Unit Testing tools are used for this reason i.e. they would help in reducing the number of bugs and as well as improve the security aspect of our software design. These tools test the source code.

❖ Tools for Unit Testing:

Criteria For Choosing Unit Testing Tools:

Some of the most important criteria for choosing Unit Testing Tools are as follows:

1. **Learning Curve of the tool:** When Picking a tool one should be wary of the learning curve of the tool. From the tool's learning curve one should know whether the time one is willing to invest would be worth it or not.
2. **Stability as well as the customer support of the tool:** A good unit testing tool should have great customer support as well as online resources and documentation to help the users if they get stuck.
3. **Simple Graphical User Interface (GUI):** The tools should have an excellent reporting Interface so that clear reports once generated would help the users to conclude the test results in fewer time intervals.

❖ Tools for Unit Testing:

4. **Ability to test data input data for load testing or difficult test scenarios:** The unit testing tool should be such that it supports test data input from various types of data files such as Excel, and XML. This would help the automation Testers in a great way.
5. **Ability to maintain the test scripts:** In automation testing, the Record and playback feature is extremely important and the ability of a tool to maintain and record scripts would be a great solution for the Automation testers.
6. **Types of Testing supported:** The unit testing tool should be such that it could support maximum testing types namely Unit, regression, etc. The tool should be good enough for automating our complex test case requirements.
7. **Allocated Budget:** Above all, the budget should be carefully considered before coming to a decision.

❖ Tools for Unit Testing:

Following are some of the unit testing tools:

1. LambdaTest
2. JUnit
3. NUnit
4. TestNG
5. PHPUnit
6. Mockito
7. Cantata
8. Karma
9. Mocha
10. TypeMock.

❖ Tools for Unit Testing:

Following are some of the unit testing tools:

1. LambdaTest
2. JUnit
3. NUnit
4. TestNG
5. PHPUnit
6. Mockito
7. Cantata
8. Karma
9. Mocha
10. TypeMock.